

CHƯƠNG 3:

XỬ LÝ THÔNG TIN TRÊN MÁY TÍNH:

LẬP TRÌNH CƠ SỞ DỮ LIỆU

Khoa Khoa học và kỹ thuật thông tin
Bộ môn Thiết bị di động và Công nghệ Web

NỘI DUNG

1. Stored Procedure.
2. Trigger.
3. Function.
4. Cursor.

Lập trình Procedure

Giới thiệu

- Một Stored Procedure (Thủ tục được lưu trữ) là bao gồm các câu lệnh Transact-SQL và được lưu lại trong cơ sở dữ liệu.
- Để thực thi chỉ cần gọi ra.
- Transact-SQL (T-SQL) là một ngôn ngữ lập trình được sử dụng làm trung gian giữa cơ sở dữ liệu và các ứng dụng. Nó tương đối dễ học vì thực chất nó được tạo bởi hầu hết là các lệnh SQL.

Lợi ích của Stored Procedure

- Module hóa: Chỉ cần viết Stored Procedure 1 lần, sau đó có thể gọi nó nhiều lần ở trong ứng dụng.
- Thực thi nhanh hơn: Stored Procedure sẽ được biên dịch và lưu vào bộ nhớ khi được tạo ra - thực thi nhanh hơn so với việc gửi từng đoạn lệnh SQL tới SQL Server.
- Giảm tải băng thông: gom các câu lệnh SQL vào 1 Stored Procedure và chỉ phải gọi đến 1 lần duy nhất qua mạng thay vì phải gọi nhiều lần.

PROCEDURE không có tham số

— Khai báo một stored procedure không có tham số:

```
CREATE PROCEDURE procedure_name
AS
BEGIN
    sql_statement
END
GO
```

— Thực thi stored procedure:

```
EXEC procedure_name
```

Ví dụ

Cho bảng Customers:

CustomerID	CustomerName	ContactName	Address	City	PostalCode	Country
1	Alfreds Futterkiste	Maria Anders	Obere Str. 57	Berlin	12209	Germany
2	Ana Trujillo Emparedados y helados	Ana Trujillo	Avda. de la Constitución 2222	México D.F.	05021	Mexico
3	Antonio Moreno Taquería	Antonio Moreno	Mataderos 2312	London	05023	Mexico
4	Around the Horn	Thomas Hardy	120 Hanover Sq.	London	WA1 1DP	UK
5	Berglunds snabbköp	Christina Berglund	Berguvsvägen 8	Luleå	S-958 22	Sweden

Ví dụ

- Viết procedure liệt kê danh sách tất cả các khách hàng.

```
CREATE PROCEDURE GetCustomersList  
AS  
    SELECT * FROM Customers  
GO
```

- Thực thi PROCEDURE:

```
EXEC GetCustomersList
```


PROCEDURE có tham số

- Ta có thể truyền vào các tham số đầu vào cho một Procedure.
Một Procedure có thể có 1 hoặc nhiều tham số.
- Có 3 trường hợp tham số cho Procedure là:
 - + Một tham số vào (input).
 - + Nhiều tham số vào (multiple input).
 - + Tham số ra (output).

Cú pháp khai báo Procedure có tham số

— Khai báo một stored procedure có tham số:

```
CREATE PROCEDURE procedure_name <@tham số 1 kiểu  
tham số 1, ... , @tham số n kiểu tham số n>  
AS
```

```
BEGIN
```

```
    sql_statement
```

```
END
```

```
GO
```

— Thực thi stored procedure:

```
EXEC procedure_name
```

Ví dụ

CustomerID	CustomerName	ContactName	Address	City	PostalCode	Country
1	Alfreds Futterkiste	Maria Anders	Obere Str. 57	Berlin	12209	Germany
2	Ana Trujillo Emparedados y helados	Ana Trujillo	Avda. de la Constitución 2222	México D.F.	05021	Mexico
3	Antonio Moreno Taquería	Antonio Moreno	Mataderos 2312	London	05023	Mexico
4	Around the Horn	Thomas Hardy	120 Hanover Sq.	London	WA1 1DP	UK
5	Berglunds snabbköp	Christina Berglund	Berguvsvägen 8	Luleå	S-958 22	Sweden

Ví dụ trường hợp 1 tham số

— Khai báo Procedure:

```
CREATE PROCEDURE GetCustomersByCity @City nvarchar(30)
AS
    SELECT * FROM Customers WHERE City = @City
GO
```

— Gọi thực thi Procedure:

```
EXEC GetCustomersByCity @City = 'London'
```

Ví dụ trường hợp nhiều tham số

— Khai báo procedure:

```
CREATE PROCEDURE GetCustomersByCityPostCode @City  
nvarchar(30), @PostalCode nvarchar(10)  
AS  
    SELECT * FROM Customers WHERE City = @City AND  
    PostalCode = @PostalCode  
GO
```

— Thực thi procedure:

```
EXEC GetCustomersByCityPostCode @City = 'London',  
@PostalCode = 'WA1 1DP'
```

Cú pháp khai báo Procedure có tham số ra

— Khai báo một stored procedure có tham số:

```
CREATE PROCEDURE procedure_name <@tham số 1 kiểu  
tham số 1, tham số 2 kiểu tham số 2 OUTPUT, ...>  
AS
```

```
BEGIN
```

```
    sql_statement
```

```
END
```

```
GO
```

— Thực thi stored procedure:

```
EXEC procedure_name
```

Ví dụ trường hợp tham số ra

— Khai báo procedure:

```
CREATE PROCEDURE GetCustomerAddress @CustomerName
nvarchar(50), @CustomerAddress nvarchar(100) OUTPUT
AS
    SELECT @CustomerAddress=Address FROM Customers WHERE
    CustomerName = @CustomerName
GO
```

— Thực thi procedure:

```
DECLARE @CustAddress nvarchar(100)
EXEC GetCustomerAddress @CustomerName = 'Maria Anders',
@CustomerAddress = @CustAddress OUTPUT
PRINT @CustAddress
```

BÀI TẬP DÙNG STORED PROCEDURE

NHANVIEN (MANV, HONV, TENLOT, TENNV, NGSINH, DCHI, PHAI, LUONG, MA_NQL, PHG)

PHONGBAN (MAPHG, TENPHG, TRPHG, NG_NC)

DIADIEM (MADIADIEM, TENDIADIEM)

DIADIEM_PHG (MADIADIEM, MAPHG)

THANNHAN (MA_NVIEN, TENTN, PHAI, NGSINH, QUANHE)

DEAN (MADA, TENDA, DDIEM_DA, PHONG)

PHANCONG (MA_NVIEN, MADA, THOIGIAN)

BÀI TẬP DÙNG STORED PROCEDURE

1. Tìm các đề án theo tên địa điểm cụ thể
2. Tìm các nhân viên có lương trong khoảng từ a đến b
3. Tính tổng thời gian tham gia các đề án của một nhân viên cụ thể
4. Tìm mã, họ tên và ngày nhận chức của trưởng phòng của một phòng ban cụ thể
5. Tìm họ tên, địa chỉ và số lượng thân nhân của một nhân viên cụ thể

BÀI TẬP

1. Tìm các đề án theo tên địa điểm cụ thể

```
CREATE PROCEDURE SP_DeAnTheoDiaDiem @TenDiaDiem NVARCHAR(50)
AS
BEGIN
    SELECT MADA, TENDA, PHONG
    FROM DEAN, DIADIEM
    WHERE DDIEM_DA = MADIADIEM AND TENDIADIEM = @TenDiaDiem
END
GO
-- Thực thi
EXEC SP_DeAnTheoDiaDiem @TenDiaDiem = N'TP.Hồ Chí Minh'
```

BÀI TẬP

2. Tìm các nhân viên có lương trong khoảng từ a đến b

```
CREATE PROCEDURE SP_NhanVienTheoLuong @LuongTu INT, @LuongDen INT  
AS
```

```
BEGIN
```

```
    SELECT MANV, HONV, TENLOT, TENNV, LUONG
```

```
    FROM NHANVIEN
```

```
    WHERE LUONG BETWEEN @LuongTu AND @LuongDen
```

```
END
```

```
GO
```

```
-- Thực thi
```

```
EXEC SP_NhanVienTheoLuong @LuongTu = 5000, @LuongDen = 15000
```

BÀI TẬP

3. Tính tổng thời gian tham gia các đề án của một nhân viên cụ thể

```
CREATE PROCEDURE SP_TongTGThamGiaDeAn
```

```
    @MaNV INT, @TongTG INT OUTPUT
```

```
AS
```

```
BEGIN
```

```
    SELECT @TongTG = SUM(THOIGIAN) FROM PHANCONG
```

```
    WHERE MA_NVIENT = @MaNV
```

```
END
```

```
GO
```

```
-- Thực thi
```

```
DECLARE @Tong INT
```

```
EXEC SP_TongTGThamGiaDeAn @MaNV = 101, @TongTG = @Tong OUTPUT
```

```
PRINT N'Tổng thời gian tham gia đề án: ' + CAST(@Tong AS NVARCHAR)
```

BÀI TẬP

4. Tìm mã, họ tên và ngày nhận chức của trưởng phòng của một phòng ban cụ thể

```
CREATE PROCEDURE SP_ThongTinTruongPhong  
    @MaPhong INT, @MaTruongPhong INT OUTPUT,  
    @TenTruongPhong NVARCHAR(40) OUTPUT, @NgayNhanChuc DATE OUTPUT  
AS  
BEGIN  
    SELECT @MaTruongPhong = TRPHG,  
           @TenTruongPhong = HONV + ' ' + TENLOT + ' ' + TENNV,  
           @NgayNhanChuc = NG_NC FROM PHONGBAN, NHANVIEN  
WHERE TRPHG = MANV AND MAPHG = @MaPhong  
END GO
```

BÀI TẬP

4. Tìm mã, họ tên và ngày nhận chức của trưởng phòng của một phòng ban cụ thể

-- Thực thi

```
DECLARE @MaTP INT, @TenTP NVARCHAR(40), @NgayNC DATE
```

```
EXEC SP_ThongTinTruongPhong
```

```
    @MaPhong = 2,
```

```
    @MaTruongPhong = @MaTP OUTPUT,
```

```
    @TenTruongPhong = @TenTP OUTPUT,
```

```
    @NgayNhanChuc = @NgayNC OUTPUT
```

```
PRINT N'Mã trưởng phòng: ' + CAST(@MaTP AS NVARCHAR)
```

```
PRINT N'Tên trưởng phòng: ' + @TenTP
```

```
PRINT N'Ngày nhận chức: ' + CAST(@NgayNC AS NVARCHAR)
```

BÀI TẬP

5. Tìm họ tên, địa chỉ và số lượng thân nhân của một nhân viên cụ thể

```
CREATE PROCEDURE SP_ThongTinNVvaSoThanNhan  
    @MaNV INT, @HoTen NVARCHAR(100) OUTPUT,  
    @DiaChi NVARCHAR(100) OUTPUT, @SoThanNhan INT OUTPUT  
AS  
BEGIN  
    SELECT @HoTen = HONV + ' ' + TENLOT + ' ' + TENNV, @DiaChi = DCHI  
    FROM NHANVIEN WHERE MANV = @MaNV  
    SELECT @SoThanNhan = COUNT(*)  
    FROM THANNHAN WHERE MA_NVIENTHANNHAN = @MaNV  
END GO
```

BÀI TẬP

5. Tìm họ tên, địa chỉ và số lượng thân nhân của một nhân viên cụ thể

-- Thực thi

```
DECLARE @Ten NVARCHAR(100), @DC NVARCHAR(100), @SLTN INT
EXEC SP_ThongTinNVvaSoThanNhan @MaNV = 102,
    @HoTen = @Ten OUTPUT,
    @DiaChi = @DC OUTPUT,
    @SoThanNhan = @SLTN OUTPUT

PRINT N'Họ tên: ' + @Ten
PRINT N'Địa chỉ: ' + @DC
PRINT N'Số lượng thân nhân: ' + CAST(@SLTN AS NVARCHAR)
```


Lập trình Trigger

Giới thiệu

- **Trigger** là **Stored Procedure** đặc biệt sẽ **tự động** thực hiện khi có hành động bổ sung dữ liệu lên 1 table mà trigger bảo vệ. Trigger có thể bao gồm hầu hết các phát biểu T-SQL.

Các tính chất của Trigger

- Liên kết với Table.
 - + Trigger được định nghĩa trên 1 table cụ thể gọi là Trigger liên kết Table.
- Thực hiện tự động.
 - + Nếu có các hành động INSERT, UPDATE, DELETE mà trigger định nghĩa nó sẽ tự động thực hiện.
 - + Nó không được gọi trực tiếp và không chấp nhận tham số.

Các tính chất của Trigger (tt)

— Là 1 giao tác (Transaction):

- + Trigger và phát biểu tạo ra nó được thực hiện như là 1 giao tác , có thể Rollback bất cứ đâu trong trigger. Trong Trigger có thể có Rollback Trans mà không có Begin Trans, SQL Server tự hiểu có Begin Trans ảo ở đây.
- + Nếu có Rollback và nó được thực hiện thì toàn Transaction sẽ Rollback.
- + Nếu trong 1 batch có Rollback và nó được thực hiện thì toàn batch sẽ bị hủy các phát biểu sau Rollback sẽ không thực hiện.

— Nên tránh lạm dụng Rollback trong Trigger vì nó phải Làm undo các thao tác trước đó → Kiểm tra tính hợp lệ trước khi bắt đầu transaction

Lợi ích của Trigger

- Bổ sung dây chuyền (cascade).
 - + Các bổ sung dây chuyền bằng Trigger làm giảm ược các coding cần thiết cho sự thay đổi trên các bảng liên quan.
- Làm tăng cường toàn vẹn dữ liệu.
 - + Do có thể tham chiếu nhiều cột trên các bảng nên có tác dụng hơn check. Tuy vậy các ràng buộc ưu tiên kiểm tra trước, nếu vi phạm trigger không thực thi.
- Thông báo lỗi do người dùng định nghĩa.
 - + Các thông báo Rule, Default, Check
- Bảo quản dữ liệu không tiêu chuẩn hóa.

Bảo quản dữ liệu không tiêu chuẩn hóa

Ví dụ:

HOADON (MSHD, NGAYHD, **TONGTIEN**)

CTHOADON (MSHD, MSMH, **SOLUONG**, **DONGIA**)

Khi sửa số lượng, đơn giá dẫn đến ảnh hưởng tổng tiền.

Lưu ý

- Trigger là phản ứng, ràng buộc được thực hiện trước.
- Ràng buộc được kiểm tra đầu tiên.
- 1 Bảng có thể viết nhiều trigger.
- Trigger không thể tạo trên View và bảng tạm.
- Trigger không thể trả về các tập kết quả.

Các loại Trigger phổ biến

— AFTER Trigger (FOR Trigger):

Được thực thi ngay sau khi câu lệnh INSERT, UPDATE hoặc DELETE xảy ra.

— INSTEAD OF Trigger:

Thực thi thay thế cho các câu lệnh INSERT, UPDATE, DELETE gốc, trước khi dữ liệu thực sự thay đổi.

Các thao tác với trigger

- Tạo trigger.
- Sửa trigger.
- Xoá trigger.

Tạo trigger

```
Create Trigger TenTrigger
On table [with (...)]
[For / Instead of insert, update, delete]
    [With append]
    [Not for Relication]
As
    phát biểu SQL
```

Sửa trigger

Alter Trigger TenTrigger
On Tentable [with encryption]
[for / instead of *inser, update, delete*]
[not fro replication]
AS

Phát biểu SQL

Xoá trigger

- Cú pháp: Drop Trigger TenTrigger
- Có thể làm mất hiệu lực tạm thời của Trigger lên 1 table
- Cú pháp :

```
Alter Table    tenTable  
    {Enable/ Disable } Trigger {all/TenTrigger}
```

- Vd :

```
Alter      Table  KETQUA  
    Disable Trigger  KiemTraThi2Lan
```

- Muốn có hiệu lực lại : dùng **Enable**

Các tính chất của trigger

- Khi tạo Trigger, thông tin trigger được insert vào các bảng hệ thống sysobject và syscomment.
- Có sẵn hai bảng đặc biệt trong các trigger là: **deleted** và **inserted**.
- Bảng **deleted** chứa các bản sao các dòng bị tác động bởi các phát biểu **Update và delete**.
- Bảng **inserted** chứa các bản sao các dòng bị tác động bởi các phát biểu **insert và Update**.
- Không thể thay đổi dữ liệu trên các bảng deleted và inserted trực tiếp, có thể dùng phát biểu select.

Chú ý

- SQL không cho phép các phát biểu sau dùng Trigger :
 - + Tất cả các phát biểu create, alter, drop.
 - + Grant, Revoke, Deny.
 - + Load, restore.
 - + Reconfigure
 - + Truncate Table.
 - + Update Statistic.
 - + Select into.

Ví dụ 1: Tự động lưu lịch sử khi thay đổi lương của nhân viên.

Cho bảng: LICHSULUONG(MANV, LUONG_CU, LUONG_MOI, NGAYTHAYDOI)

```
CREATE TRIGGER TRG_LuuLichSuLuong
ON NHANVIEN FOR UPDATE
AS
BEGIN
    INSERT INTO LICHSULUONG(MANV, LUONG_CU, LUONG_MOI, NGAYTHAYDOI)
    SELECT i.MANV, d.LUONG, i.LUONG, GETDATE()
    FROM inserted i -- dữ liệu mới
    JOIN deleted d -- dữ liệu cũ
    ON i.MANV = d.MANV
    WHERE i.LUONG <> d.LUONG -- chỉ lưu khi lương thay đổi
END
```

Ví dụ 2: Ngăn không cho xóa nhân viên nếu họ đang tham gia đề án

```
CREATE TRIGGER TRG_NganXoaNhanVien
ON NHANVIEN INSTEAD OF DELETE AS
BEGIN
    IF EXISTS (SELECT * FROM PHANCONG
               WHERE MA_NVIEIN IN (SELECT MANV FROM deleted))
    BEGIN
        RAISERROR (N'Không được xóa nhân viên đang tham gia đề án!',
16, 1)
        ROLLBACK TRANSACTION
    END
    ELSE    DELETE FROM NHANVIEN WHERE MANV IN (SELECT MANV FROM deleted)
END
```


Các hoạt động của trigger.

- Hoạt động khi insert.
- Hoạt động khi update.
- Hoạt động khi delete

Hoạt động khi insert

- Phát biểu Insert thực hiện trên bảng Trigger định nghĩa.
- **Phát biểu Insert được ghi .**
- Trigger bị bắn phá và phát biểu trong Trigger thực thi khi Trigger Insert bắn phá, các dòng mới được thêm vào hai bảng Trigger và Insert. Bảng Insert nắm giữ một bản sao các dòng đã Insert. Bảng Insert chứa các hoạt động từ phát biểu trên. Trigger có thể khảo sát trên bản Insert để quyết định hành động của mình. Các dòng trong bảng Insert thường bị trùng lặp.

Hoạt động khi update

- Phát biểu Update thực hiện trên bản Trigger định nghĩa.
- Phát biểu Update được ghi lại.
- Trigger bị bắn phá và phát biểu trong trigger thực hiện có thể xem như 2 bước :
 - + Xóa (delete)
 - + Chèn (insert)
- Như thế các dòng gốc sẽ di chuyển đến các bảng deleted và các dòng cập nhật được chèn vào bảng inserted.

Hoạt động khi update

- Có thể định nghĩa 1 trigger để giám sát việc cập nhật DL trên 1 cột đặc biệt bằng cách dùng phát biểu **if update**, nó cho phép thực hiện khi có sự cập nhật trên các cột đã chỉ định.
- Cú pháp: If update (column) [{AND | OR} update (**cols**)
- VD:

Create Trigger **CapnhatMSSV**

If update (MSSV)

Begin

Print 'Không được cập nhật mã số SV'

Rollback transaction

End

Hoạt động khi Delete

- Phát biểu Delete thực hiện trên bản Trigger định nghĩa.
- Phát biểu Delete được ghi lại.
- Trigger bắn phá và các phát biểu trong Trigger thực thi.
- Khi Trigger bắn phá các dòng bị xóa được đặt trong bản Deleted. Bảng Deleted lưu giữ một bản sao các dòng bị xóa.
- Chú ý:
 - + Các dòng Deleted không tồn tại trong cơ sở dữ liệu, vì vậy bản xóa và cơ sở dữ liệu không có các “dòng chung”
 - + Trigger bắn phá bởi Delete không thực hiện phát biểu: TRUNCATE TABLE

Ví dụ 3

Tự động cập nhật tổng tiền (TONGTIEN) trong bảng HOADON mỗi khi có thêm, sửa hoặc xóa chi tiết hóa đơn (CTHD), đồng thời in thông báo giả lập gửi email.

```
CREATE TRIGGER TRIG_CTHD
ON CTHD
AFTER INSERT, UPDATE, DELETE
AS
BEGIN
    UPDATE HOADON
    SET TONGTIEN = (
        SELECT COALESCE(SUM(SOLUONG * DONGIA), 0)
        FROM CTHD
        WHERE CTHD.SOHD = HOADON.SOHD
    )
    WHERE SOHD IN (
        SELECT SOHD FROM INSERTED
        UNION
        SELECT SOHD FROM DELETED
    )
    -- Send email
    PRINT 'Email notification sent to customer'
END
```

Các dạng đặc biệt của Trigger

- Trigger lồng.
- Trigger đệ quy.

Trigger lồng

- Trigger có thể lồng nhau **32 mức**. Bất cứ trigger nào trong chuỗi lồng bị loop (mức lồng vượt quá mức 32) transaction sẽ **rollback**.
- **Chú ý:**
 - + Mặc định, cấu hình lồng bằng ON.
 - + Trigger bị lồng không bị bắn phá 2 lần trong 1 transaction, mặt khác, trigger không thể tự bắn phá chính nó. Trong trường hợp này ta nói trigger đệ quy, sẽ bàn sau.
 - + Một trigger là 1 transaction, 1 lỗi xảy ra tại bất kỳ ở đâu, tất cả việc bổ sung DL sẽ rollback, có thể thêm phát biểu print để kiểm tra lỗi xảy ra tại đâu.

Trigger lồng

- Mức lồng sẽ tăng lên khi có 1 trigger lồng bị bắn phá. Để tránh mức lồng vượt quá mức 32, nên dùng hàm @@nestlevel.

Lợi ích của trigger lồng

- Dùng trigger là công cụ hữu hiệu đảm bảo DL toàn vẹn
- Khi install SQL Server, lồng nhau là default. Có thể làm mất tạm thời cho phép trigger lồng bằng cú pháp:
 - + `sp_configure 'nested Triggers', 0.`
 - + Muốn ngược lại: `sp_configure 'nested Triggers', 1`
- Ví dụ: Có thể thiết lập trigger khi một đơn đặt hàng mới được đặt. trigger này sẽ gọi một trigger khác gửi thông báo qua email cho các bên liên quan và một trigger khác cập nhật trạng thái của đơn đặt hàng trong cơ sở dữ liệu.

Điểm yếu của trigger lồng

- Làm mất tính log của trigger do Tính phức tạp của nó.
- Có thể thay thế chức năng log của trigger bằng con đường *mỗi trigger được khởi tạo sẽ bổ sung tất cả dữ liệu cần thiết* nhờ trợ giúp của **Stored Procedure**.

Trigger đệ quy

- Trigger có thể có các phát biểu Update, insert, delete đến cùng một bảng hay bảng khác. Khi option đệ quy bật lên trigger có thể thay đổi table mà bản phá chính nó. Execute option đệ quy mặc định là không, có thể thay đổi VLOOKUP.
- Cú pháp:
Sp-dboption database, 'recursive triggers', (True False)

Các loại đệ quy

— Đệ quy trực tiếp:

+ Ví dụ: một ứng dụng cập nhật table A, gây nên trig1. Trig1 lại cập nhật A 1 lần nữa, và trig1 bị gọi.

— Đệ quy gián tiếp:

+ Ví dụ: một ứng dụng cập nhật table A, gây nên trig1 bắn phá, trig1 cập nhật table B, gây nên trig2, trig2 cập nhật table A, trig1 lại bắn phá 1 lần nữa

BÀI TẬP DÙNG TRIGGER

1. Tạo ràng buộc không cho phép giảm lương nhân viên
2. Tạo ràng buộc không cho phép xóa phòng ban nếu phòng ban còn nhân viên
3. Tạo ràng buộc một nhân viên tham gia không quá 3 đề án

BÀI TẬP

1. Tạo ràng buộc không cho phép giảm lương nhân viên

```
CREATE TRIGGER TRG_NganGiamLuong_Update
ON NHANVIEN
FOR UPDATE
AS
BEGIN
    IF EXISTS (SELECT * FROM INSERTED i JOIN DELETED d ON i.MANV = d.MANV
               WHERE i.LUONG < d.LUONG)
    BEGIN
        RAISERROR(N'Không được giảm lương nhân viên!', 16, 1)
        ROLLBACK TRANSACTION
    END
END
```

BÀI TẬP

2. Tạo ràng buộc không cho phép xóa phòng ban nếu phòng ban còn nhân viên

```
CREATE TRIGGER TRG_NganXoaPhongBan_Delete
```

```
ON PHONGBAN INSTEAD OF DELETE
```

```
AS
```

```
BEGIN
```

```
    IF EXISTS (SELECT * FROM NHANVIEN WHERE PHG IN (SELECT MAPHG FROM DELETED))
```

```
        BEGIN
```

```
            RAISERROR (N'Phòng ban vẫn còn nhân viên, không thể xóa!',16,1)
```

```
            ROLLBACK TRANSACTION
```

```
        END
```

```
    ELSE
```

```
        DELETE FROM PHONGBAN WHERE MAPHG IN (SELECT MAPHG FROM DELETED)
```

```
END
```


BÀI TẬP

3. Tạo ràng buộc một nhân viên tham gia không quá 3 đề án

```
CREATE TRIGGER TRG_GioiHanSoDeAnNV
ON PHANCONG AFTER INSERT, UPDATE
AS
BEGIN
    IF EXISTS (SELECT i.MA_NVIENT FROM INSERTED I JOIN PHANCONG p
               ON i.MA_NVIENT = p.MA_NVIENT
               GROUP BY i.MA_NVIENT HAVING COUNT(p.MADA) > 3)
    BEGIN
        RAISERROR (N'Mỗi nhân viên chỉ được tham gia tối đa 3 đề án.',16,1)
        ROLLBACK TRANSACTION
    END
END
```

Lập trình Function

Giới thiệu

- Function (Hàm) là một đối tượng trong cơ sở dữ liệu bao gồm một tập nhiều câu lệnh được nhóm lại với nhau và được tạo ra với mục đích sử dụng lại.
- Trong SQL Server, hàm được lưu trữ và bạn có thể truyền các tham số vào cũng như trả về các giá trị.

Các thao tác với hàm

- Khai báo hàm trong SQL:
 - + Dạng 1: Scalar-valued Functions.
 - + Dạng 2: Inline Table-valued Functions.
 - + Dạng 3: Multi-statement Table-valued Functions.
- Xoá hàm.

Cú pháp: `DROP FUNCTION function_name;`

Cú pháp Dạng 1:

Scalar-valued Functions

```

CREATE FUNCTION [ owner_name. ] function_name
    ( [ { @parameter_name [AS] scalar_parameter_data_type [
= default ] } [ ,...n ] ] )
RETURNS scalar_return_data_type
    [ AS ]
BEGIN
    function_body
    RETURN scalar_expression
END
    
```

Ví dụ 1

— Thực hiện phép tính cộng hai số nguyên:

— Khai báo:

```
CREATE FUNCTION CONG (@X1 INT, @X2 INT) RETURNS INT
```

```
AS
```

```
BEGIN
```

```
    RETURN @X1+@X2
```

```
END
```

— Sử dụng

```
SELECT DBO.CONG (1, -5) AS KQ
```

Cú pháp Dạng 2:

Inline Table-valued Functions

```
CREATE FUNCTION [ owner_name. ] function_name
    ( [ { @parameter_name [AS] scalar_parameter_data_type [
= default ] } [ ,...n ] ] )
RETURNS TABLE
    [ AS ]
RETURN [ ( ] select-stmt [ ) ]
```

Ví dụ 2

— Tìm sinh viên có điểm thấp hơn một điểm cho trước:

— Khai báo:

```
CREATE FUNCTION DIEMNHOHON(@DIEM INT) RETURNS TABLE  
AS  
    RETURN (SELECT * FROM KETQUA WHERE DIEM < @DIEM )
```

— Sử dụng

```
SELECT * FROM DBO.DIEMNHOHON(5)
```


Cú pháp Dạng 3:

Multi-statement Table-valued Functions

```

CREATE FUNCTION [ owner_name. ] function_name
    ( [ { @parameter_name [AS] scalar_parameter_data_type [
= default ] } [ ,...n ] ] )
RETURNS @return_variable TABLE < table_type_definition >
[ WITH < function_option > [ [,] ...n ] ]
[ AS ]
BEGIN
    function_body
RETURN
END
    
```

Ví dụ 3

```
CREATE FUNCTION KETQUATHEODIEM(@DIEM INT) RETURNS  
@RESULT TABLE (MSGV INT, MSMH INT, DIEM INT)  
AS  
BEGIN  
    INSERT INTO @RESULT (MSGV, MSMH, DIEM)  
    SELECT MSGV, MSMH, DIEM FROM KETQUA WHERE  
    DIEM > @DIEM  
    RETURN  
END
```

SO SÁNH SP VÀ FUNCTION

STORED PROCEDURE	FUNCTION
Là một tập các câu lệnh SQL	
Có thể trả về 0, 1 hoặc nhiều giá trị	Luôn trả về một giá trị hoặc bảng dữ liệu
Có tham số vào và ra	Chỉ có các tham số vào
Có thể gọi Function trong Stored Procedure	Không thể gọi Stored Procedure trong Function
Không thể sử dụng trong câu SELECT / WHERE / HAVING	Có thể sử dụng trong câu SELECT / WHERE / HAVING / JOIN
Có TRY-CATCH	Không có TRY-CATCH
Có Transaction	Không có Transaction

Lập trình Cursor

Giới thiệu

- **Cursor** (con trỏ) là một kỹ thuật lập trình CSDL cao cấp trong SQL. Dùng tính toán và chọn một trường ô (duyệt – traversal) trong một bảng (Table) trong CSDL.
- Các cursor tạo điều kiện xử lý tiếp theo kết hợp với việc traversal, chẳng hạn như thu hồi, bổ sung và loại bỏ các bản ghi cơ sở dữ liệu.
- Trong các thủ tục SQL, một cursor sẽ làm cho nó có thể định nghĩa một tập kết quả (một tập hợp các dòng dữ liệu) và thực hiện logic phức tạp trên cơ sở hàng bằng hàng.

Vấn đề

- Các câu lệnh trong SQL đều thao tác lên nhiều dòng dữ liệu thỏa điều kiện WHERE cùng lúc mà không thể thao tác trên từng dòng cụ thể.
- Cursor là kiểu dữ liệu có thể duyệt qua từng dòng kết quả trả về của câu lệnh SELECT giúp chúng ta có thể xử lý khác nhau cho từng kết quả mà ta mong muốn.
- Nhưng lại tồn tại khuyết điểm là xử lý rất chậm.

Các bước sử dụng một Cursor

- Để sử dụng con trỏ trong cơ sở dữ liệu, chúng ta cần:
 1. Khai báo một con trỏ xác định một tập kết quả.
 2. Thiết lập kết quả cho con trỏ.
 3. Gán dữ liệu cho các biến cục bộ cần thiết cho con trỏ và một hàng.
 4. Đóng cursor khi hoàn thành.

Các cú pháp khai báo 1 cursor

1- Cú pháp khai báo một Cursor

```
DECLARE ten_con_tro [SCROLL] CURSOR  
FOR  
    SELECT  
    FROM  
    WHERE  
[FOR {READ ONLY | UPDATE [OF ten_cot [truong]]}]
```

Cú pháp khai báo một Cursor

- **SCROLL**: cho phép con trỏ di chuyển lên xuống, qua lại giữa các mẫu tin.
- **READ ONLY**: không cho phép thực thi các hành động như update,...
- **UPDATE**: xác định khả năng cập nhật của con trỏ, nếu OF được chỉ định thì chỉ có những cột, những trường trong danh sách được chỉnh sửa. Ngoài ra, chúng ta cũng có thể khởi tạo riêng con trỏ rồi mới gán lệnh SELECT cho con trỏ, như sau:

```
DECLARE @ten_con_tro CURSOR  
SET @ten_con_tro = CURSOR FOR SELECT...
```

2- Cú pháp mở một Cursor

OPEN [**GLOBAL**] ten_con_tro | @ten_con_tro

Lưu ý: GLOBAL là biến toàn cục.

3 – Cú pháp truy cập một con trỏ

```
FETCH [NEXT | PRIOR | FIRST | LAST | ABSOLUTE {n | @nVar} |  
RELATIVE {n | @nVar}]  
  
FROM [GLOBAL] ten_con_tro | @ten_con_tro  
  
[INTO @ten_bien[du_lieu]]
```

Cú pháp truy cập một con trỏ

- **NEXT, PRIOR, FIRST, LAST**: chỉ định cách đọc dữ liệu.
- **ABSOLUTE**: chỉ định n số dòng dữ liệu cần đọc. Nếu:
 - + $n = 0$: không có giá trị trả về.
 - + $n < 0$: xuất phát từ phần đáy dữ liệu.
 - + $n > 0$: xuất phát từ phần đỉnh dữ liệu.
- **RELATIVE** cũng giống như ABSOLUTE nhưng bắt đầu từ vị trí hiện tại.
- Ngoài ra, chúng ta còn có lệnh **@@FETCH_STATUS** để check xem hệ thống đọc dữ liệu thành công hay thất bại.

4 – Cú pháp đóng Cursor

CLOSE [GLOBAL] ten_con_tro | @ten_con_tro

hoặc

DEALLOCATE [GLOBAL] ten_con_tro | @ten_con_tro

Cú pháp đóng Cursor

- **Lưu ý:** CLOSE và DEALLOCATE sẽ có sự khác biệt. Với CLOSE chúng ta sẽ đóng cursor lại nhưng có thể tái sử dụng lại ở lần sau. Còn DEALLOCATE sẽ giải phóng hoàn toàn cursor ra khỏi bộ nhớ, vì vậy nếu có lệnh nào tham chiếu tới cursor có thể gây ra lỗi.

Ví dụ

— Tính điểm trung bình của sinh viên cho CSDL sau:

KQ		
MASV	TENMH	DIEM
s1	m1	7
s2	m1	8
s3	m1	6
s1	m3	4
s2	m3	9
s5	m4	3
s2	m4	8
s1	m4	9

SV		
MASV	TENSV	DIEMTB
s1	B	---
s2	A	---
s3	C	---
s5	D	---

Ví dụ 1

- Khai báo biến:
 - + DECLARE @A CHAR(10), @B FLOAT.
- Khai báo con trỏ:
 - + DECLARE X CURSOR FOR SELECT MASV FROM SV.
- Mở con trỏ:
 - + OPEN x.

Ví dụ 1

- Truy cập con trỏ:
 - + FETCH NEXT FROM X INTO @A
- Kiểm tra xem con trỏ đọc dữ liệu được hay không:
 - + WHILE (@@FETCHSTATUS = 0)
- Tính điểm TB dựa vào con trỏ:
 - + BEGIN
 - SELECT @B = AVG(DIEM) FROM KQ WHERE MASV = @A
 - UPDATE SV SET DIEMTB = @B WHERE MASV = @A
 - FETCH NEXT FROM X INTO @A
 - + END

Ví dụ 1

- Đóng con trỏ:
 - + CLOSE X.
 - + DEALLOCATE X.

Thực thi Cursor

— Để thực thi Cursor, ta dùng lệnh **EXEC**.

— VD:

+ **EXEC TINHDIEM**

CODE mẫu

```
CREATE PROCEDURE TINHDIEM
AS
BEGIN
    DECLARE @A CHAR(10), @B FLOAT
    DECLARE X CURSOR FOR SELECT MASV FROM SV
    OPEN X
    FETCH NEXT FROM X INTO @A
    WHILE (@@FETCHSTATUS = 0)
    BEGIN
        SELECT @B=AVG(DIEM) FROM KQ WHERE MASV = @A
        UPDATE SINHVIEN SET @B = DTB WHERE MASV = @A
        FETCH NEXT FROM X INTO @A
    END
    CLOSE X
    DEALLOCATE X
END
```

Ví dụ 2

- Tính điểm trung bình của từng sinh viên

Ví dụ 2

```
DECLARE X CURSOR FOR
    SELECT MASV, AVG(DIEM) AS DTB
    FROM KQ
    GROUP BY MASV
OPEN X
DECLARE @A CHAR(10), @B FLOAT
FETCH X INTO @A, @B
WHILE (@@FETCHSTATUS=0)
    BEGIN
        FETCH NEXT FROM X INTO @A, @B
    END
```

Tổng kết

- T-SQL là một dạng ngôn ngữ lập trình CSDL, kết hợp các nhóm lệnh SQL lại với nhau.
- Stored procedure là một dạng thủ tục, dùng để nhóm các câu lệnh SQL lại với nhau. Có 2 dạng là: không tham số và có tham số (tham số gồm 2 dạng là tham số vào và tham số ra).
- Trigger được thực thi tự động khi có hành vi thay đổi trên CSDL. Trigger không có tham số.
- Function là một dạng đối tượng nhận tham số đầu vào và trả về giá trị cụ thể. Dùng để tái sử dụng lại các lệnh SQL.
- Cursor là một kỹ thuật lập trình nâng cao, cho phép truy xuất dữ liệu phức tạp theo từng dòng và từng ô.

Bài tập cursor

Bài 1. Viết Cursor trả về **SỐ MÔN HỌC** của SV

Bài 2. Viết Cursor trả về **SỐ MÔN HỌC LẠI** của SV. (Sinh viên học lại khi **DIEM** < 5)

Bài 3. Viết Cursor cập nhật số môn học của sinh viên từ bảng SV vào bảng KQ.

TÀI LIỆU THAM KHẢO

1. Nguyễn Gia Tuấn Anh, Trương Châu Long, *Bài tập và bài giải SQL Server*, NXB Thanh niên (2005).
2. Đỗ Phúc, Nguyễn Đăng Ty, *Cơ sở dữ liệu*, NXB Đại học quốc gia TP HCM (2010).
3. Nguyễn Gia Tuấn Anh, Mai Văn Cường, Bùi Danh Hường, *Cơ sở dữ liệu nâng cao*, NXB Đại học quốc gia TP HCM (2019).
4. Itzik Ben-Gan, *Microsoft SQL Server 2012- TSQL Fundamentals*.

